

Curso de:

Desarrollo en React JS

Módulo 1:

Nivelación, Javascript, configuración de entorno

Unidad 3:

Javascript avanzado



Presentación

En el ecosistema del desarrollo web moderno, la capacidad de trabajar eficientemente con operaciones asincrónicas y estructuras avanzadas como clases y arrays es indispensable. Esto se debe a que las aplicaciones actuales interactúan constantemente con servidores, APIs externas, archivos y otras fuentes de datos que no están disponibles de forma inmediata. Paralelamente, exigen estructuras de código organizadas, reutilizables y escalables para garantizar su mantenimiento a largo plazo.

Para ello, JavaScript ofrece un modelo asincrónico no bloqueante que permite gestionar múltiples tareas sin detener el flujo principal de ejecución. A través de herramientas clave como los temporizadores, las promesas y la sintaxis **async/await**, es posible desarrollar aplicaciones más rápidas, reactivas y confiables. A esto se suma la comprensión del **Event Loop**, el manejo de errores asincrónicos y las buenas prácticas para evitar bloqueos o fallos silenciosos.

Al mismo tiempo, la programación orientada a objetos en JavaScript permite estructurar el código en torno a entidades lógicas que encapsulan propiedades y comportamientos. Así, mediante clases, constructores, herencia y encapsulamiento, se promueve la creación de sistemas más comprensibles, reutilizables y robustos.

Complementariamente, los métodos avanzados de arrays como **map**, **filter**, **reduce**, **find** o **forEach** resultan esenciales para manipular datos de forma

eficiente y declarativa. Estas herramientas permiten transformar, buscar, filtrar o recorrer colecciones de datos sin recurrir a estructuras imperativas complejas, facilitando así la escritura de código más expresivo y profesional.

En definitiva, esta unidad proporciona los fundamentos conceptuales y técnicos para integrar asincronía, objetos y manipulación de datos en el desarrollo de componentes interactivos, peticiones HTTP eficientes y soluciones de *software* modernas, mantenibles y alineadas con los estándares actuales de la industria.



Objetivos

Que los participantes logren...

- **Comprender** la diferencia entre ejecución síncrona y asíncrona en JavaScript y su impacto en el flujo del programa.
- **Utilizar** temporizadores, promesas y funciones `async/await` para gestionar operaciones asíncronas de manera eficiente.
- **Crear** promesas personalizadas y aplicar buenas prácticas para escribir código asíncrono legible, escalable y robusto.
- **Identificar** errores comunes en la gestión de asincronía y aplicar estrategias para evitarlos.
- **Comprender** los fundamentos de la programación orientada a objetos en JavaScript mediante clases, constructores, métodos y herencia.
- **Modelar** estructuras reutilizables aplicando encapsulamiento y principios de diseño orientado a objetos.
- **Dominar** los métodos avanzados de arrays para transformar, recorrer y filtrar datos de forma declarativa.
- **Elegir** el método adecuado según el tipo de operación requerida y combinar arrays con clases para construir funcionalidades complejas.



Bloques temáticos

1. Asincronía y promesas.

- a. Sincronía vs. asincronía en JavaScript.
- b. Temporizadores: setTimeout y setInterval.
- c. Introducción a las promesas.
- d. Creación y uso de promesas personalizadas.
- e. Funciones asincrónicas con async y await.
- f. Errores comunes y buenas prácticas en asincronía.

2. Clases, métodos avanzados de arrays.

- a. Programación orientada a objetos en JavaScript.
- b. Métodos avanzados de arrays en JavaScript.

Asincronía y promesas.

Sincronía vs. asincronía en JavaScript.

JavaScript es un lenguaje de programación **monohilo**, lo que significa que su motor ejecuta las instrucciones en un único hilo de ejecución, procesando una operación a la vez en orden secuencial. En este modelo, cada línea de código debe completarse antes de pasar a la siguiente. Esta característica define lo que se conoce como **comportamiento síncrono**, donde las instrucciones se ejecutan en el orden en que fueron escritas y ninguna avanza hasta que la anterior se haya completado. Si una de estas operaciones es lenta, por ejemplo, acceder a una API, leer un archivo o esperar una respuesta del servidor, toda la ejecución se detiene hasta que finalice dicha tarea.

Este comportamiento, aunque predecible, **representa una limitación** en el entorno de las aplicaciones web modernas, donde es común que el código deba interactuar con recursos externos, manejar archivos grandes o esperar acciones del usuario sin bloquear la interfaz. Para resolver esta necesidad, JavaScript incorpora mecanismos que permiten ejecutar tareas de manera **asíncrona**, es decir, operaciones que no bloquean el flujo principal y que delegan su resultado a un momento posterior.

En un modelo **asíncrono**, JavaScript puede iniciar una operación (como hacer una consulta al servidor) y continuar ejecutando otras instrucciones mientras espera la respuesta. Cuando la operación finaliza, una función asociada (un *callback*, una promesa o una función **async**) se encarga de gestionar el resultado. Este enfoque permite crear aplicaciones más rápidas, fluidas y reactivas, que pueden atender múltiples tareas sin congelarse ni hacer esperar al usuario.

Para comprender la diferencia, veamos una representación conceptual:



```
console.log("Inicio");  
console.log("Proceso...");  
console.log("Fin");
```

La salida será:



```
Inicio  
Proceso...  
Fin
```

En este caso, cada instrucción se ejecuta una tras otra, en orden estricto.

Ejemplo de código asíncrono.



```
console.log("Inicio");  
  
setTimeout(() => {  
  console.log("Proceso...");  
}, 2000);  
  
console.log("Fin");
```

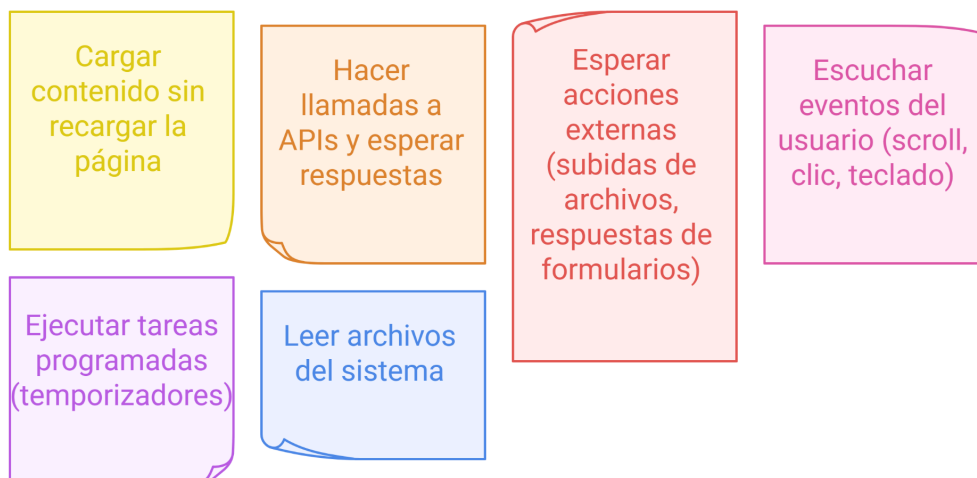
La salida será:



En este caso, `setTimeout` simula una operación que tarda 2 segundos. JavaScript no espera que termine para continuar: ejecuta la instrucción, la deja en espera, y continúa con el resto del código. Cuando pasan los 2 segundos, la función se ejecuta y muestra el mensaje. Este comportamiento es clave en la asincronía: **la tarea se programa para completarse en el futuro, sin detener el hilo principal.**

Este modelo de ejecución asincrónica es posible gracias a un conjunto de mecanismos que integran lo que se conoce como **Event Loop**, un componente interno del motor de JavaScript que coordina la ejecución de tareas pendientes, colas de mensajes y llamadas diferidas. Aunque el **Event Loop** es un concepto más avanzado que se explora en niveles superiores, es importante tener claro que permite manejar múltiples tareas "al mismo tiempo" sin salir del modelo monohilo.

La asincronía es esencial para:



En síntesis, mientras que la **sincronía** sigue un flujo de ejecución rígido y secuencial, la **asincronía** permite delegar operaciones y retomar su resultado más adelante, sin bloquear el resto del código. Esta diferencia no solo es técnica, sino también conceptual: programar con asincronía implica pensar en términos de **tiempo**, **espera**, **respuestas futuras** y **estructura no lineal**, lo que plantea desafíos pero también habilita nuevas posibilidades en el desarrollo de aplicaciones modernas.

Temporizadores: `setTimeout` y `setInterval`.

Dentro del conjunto de herramientas que JavaScript ofrece para manejar la **asincronía**, los **temporizadores** cumplen un rol fundamental: permiten **programar la ejecución de una función para el futuro**, ya sea una única vez o de forma repetida. Este tipo de operaciones no bloquea el hilo principal, sino que delega su ejecución a través del **Event Loop**, que se encargará de invocar la función cuando el tiempo especificado haya transcurrido.

En detalle, los temporizadores más utilizados en JavaScript son dos funciones globales: `setTimeout()` y `setInterval()`. Aunque comparten ciertas similitudes en cuanto a su sintaxis y propósito general, su comportamiento es distinto y responde a diferentes necesidades dentro de una aplicación.

`setTimeout()` – Ejecutar una función una vez, después de un tiempo

El método `setTimeout()` permite programar una función para que se ejecute **una única vez** después de un período de tiempo determinado (en milisegundos). En general, es útil cuando se desea **retrasar** la ejecución de una acción sin bloquear el flujo del programa.

Conceptualmente, `setTimeout` acepta dos parámetros principales:

1. Una función (*callback*) que se ejecutará en el futuro.
2. El tiempo de espera en milisegundos.

Por ejemplo, si se quiere mostrar un mensaje 3 segundos después de que el usuario haya ingresado a la página, se puede utilizar:

```
setTimeout(() => {  
  console.log("Han pasado 3 segundos");  
}, 3000);
```

En este caso, la función se ejecutará **una sola vez** luego de 3000 milisegundos (es decir, 3 segundos), sin interrumpir el resto del código.

Por su parte, `setTimeout` devuelve un identificador numérico que puede usarse para **cancelar** la ejecución antes de que ocurra, utilizando `clearTimeout()`:

```
const id = setTimeout(() => {  
  console.log("Esto no se verá");  
}, 5000);  
  
clearTimeout(id); // Cancela la ejecución
```

Este mecanismo es útil para interfaces dinámicas que deben permitir la cancelación de procesos programados, como notificaciones, mensajes emergentes o tareas condicionales.

setInterval() – Ejecutar una función repetidamente

A diferencia de `setTimeout`, la función `setInterval()` ejecuta la función especificada **de forma periódica**, con un intervalo de tiempo constante entre cada ejecución. Es ideal para realizar tareas repetitivas como actualizar un reloj, cambiar imágenes de un carrusel o realizar sondeos a una API y su sintaxis es similar:

```
setInterval(() => {  
  console.log("Este mensaje aparece cada segundo");  
}, 1000);
```

En este caso, la función se ejecutará indefinidamente cada 1000 milisegundos, hasta que sea cancelada explícitamente con `clearInterval()`.

```
const reloj = setInterval(() => {  
  console.log(new Date().toLocaleTimeString());  
}, 1000);  
  
setTimeout(() => {  
  clearInterval(reloj); // Detiene el intervalo luego de 5 segundos  
, 5000);
```

En este ejemplo, se imprime la hora actual cada segundo, pero se detiene automáticamente después de cinco segundos gracias a la combinación de ambos temporizadores.

Consideraciones importantes sobre temporizadores.

- El tiempo especificado en `setTimeout` y `setInterval` **no garantiza** una ejecución exacta, sino un **mínimo de espera**. Si el hilo principal está ocupado (por ejemplo, con una operación intensiva de cálculo), el temporizador se ejecutará en cuanto sea posible, no necesariamente con precisión milimétrica.
- El uso excesivo de `setInterval` puede generar acumulación de tareas pendientes si el código dentro del intervalo tarda más de lo esperado en ejecutarse. En esos casos, se recomienda utilizar una combinación de `setTimeout` recursivo para tener más control.
- Ambos métodos son herramientas clave para introducir el concepto de **asincronía programada** y constituyen un punto de entrada ideal para comprender cómo funcionan operaciones diferidas sin necesidad de manejar directamente promesas o `async/await`.

Aplicaciones típicas de temporizadores

Mostrar mensajes temporales o alertas que desaparecen.

Retrasar una acción para mejorar la experiencia del usuario (por ejemplo, una animación).

Ejecutar validaciones después de cierto tiempo de inactividad del usuario.

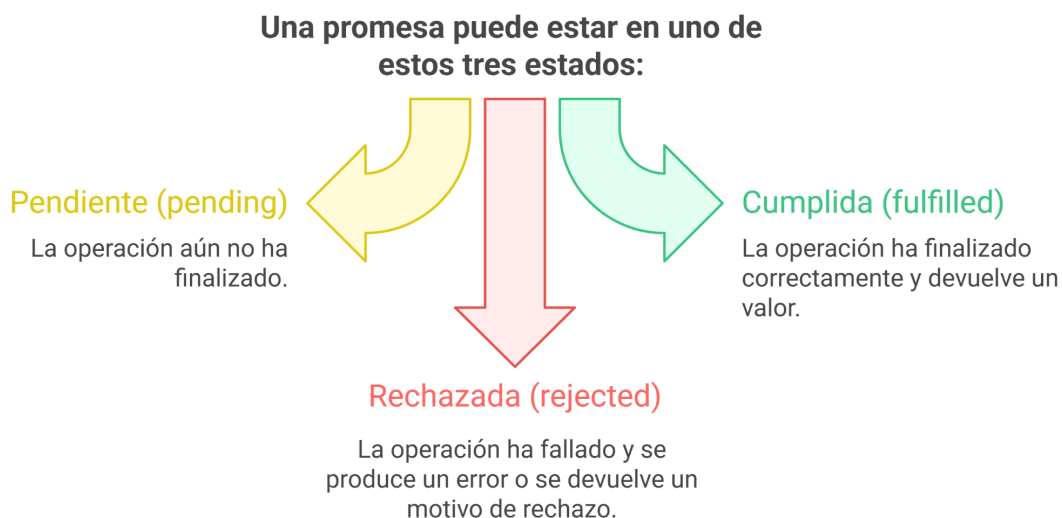
Actualizar componentes de forma periódica sin recargar la página.

Realizar tareas programadas como backups, verificaciones o sincronización de datos.

Introducción a las promesas.

A medida que las aplicaciones web se volvían más complejas, la necesidad de manejar múltiples operaciones asincrónicas de manera ordenada, predecible y legible se hizo evidente. Si bien los temporizadores y los *callbacks* ofrecían una solución básica, su uso en operaciones encadenadas o anidadas generaba lo que se conoce como ***callback hell*** o “infierno de *callbacks*”: un código difícil de leer, mantener y depurar.

Para resolver este problema, JavaScript introdujo el concepto de **promesas** (*promises*) como una abstracción para representar **el resultado futuro de una operación asincrónica**. En particular, una promesa no ejecuta la operación, sino que **representa un valor que puede estar disponible ahora, en el futuro o nunca**, y permite manejar ese resultado de forma más estructurada.



En términos generales, el principal beneficio de las promesas es que **permiten asociar funciones a ejecutar cuando la operación finaliza**, sin necesidad de bloquear el hilo principal ni de anidar múltiples *callbacks*. Este enfoque mejora la

legibilidad del código, facilita el manejo de errores y permite encadenar operaciones asíncronas de forma más clara.

Cómo se consume una promesa.

En la práctica, las promesas se utilizan ampliamente con APIs modernas del navegador (como `fetch`, `FileReader`, `Web Storage`, entre muchas otras).

Una vez que una promesa es devuelta, se la puede encadenar con los métodos `.then()` y `.catch()`:

- `.then()` se ejecuta cuando la promesa se cumple exitosamente.
- `.catch()` se ejecuta si ocurre un error o si la promesa es rechazada.

Por ejemplo, el método `fetch` (para hacer solicitudes HTTP) devuelve una promesa:

```
fetch("https://api.ejemplo.com/datos")
  .then(respuesta => respuesta.json())
  .then(data => {
    console.log("Datos obtenidos:", data);
  })
  .catch(error => {
    console.error("Error al obtener los datos:", error);
  });
```

Este ejemplo permite realizar una solicitud a una API de forma asíncrona, y define con claridad qué hacer con la respuesta y cómo manejar un posible error, sin bloquear el flujo del programa.

Ventajas del modelo de promesas.

- ✓ **Encadenamiento lógico:** las promesas permiten construir flujos asincrónicos en los que cada operación depende del resultado de la anterior.
- ✓ **Manejo centralizado de errores:** usar `.catch()` al final del encadenamiento evita la necesidad de manejar errores en cada paso.
- ✓ **Mejora la legibilidad:** a diferencia de múltiples *callbacks* anidados, el uso de promesas mantiene el código más plano y fácil de seguir.
- ✓ **Compatibilidad con `async/await`:** las promesas son la base sobre la que se construye la sintaxis moderna de asincronía con `async` y `await`, que será abordada más adelante.

Ejemplo comparativo: *callback* vs. promesa.

Con *callback*:

```
leerArchivo("datos.json", function(error, datos) {  
  if (error) {  
    console.log("Ocurrió un error");  
  } else {  
    procesar(datos, function(error, resultado) {  
      if (error) {  
        console.log("Error en el procesamiento");  
      } else {  
        console.log("Resultado final:", resultado);  
      }  
    });  
  }  
});
```

Con promesas:


```
leerArchivoPromesa("datos.json")  
  .then(datos => procesarPromesa(datos))  
  .then(resultado => console.log("Resultado final:", resultado))  
  .catch(error => console.error("Hubo un error:", error));
```

Ambos fragmentos realizan tareas similares, pero la versión con promesas es más legible, menos propensa a errores y más sencilla de extender.

La promesa como contrato.

En términos conceptuales, una promesa actúa como un **contrato**: se acuerda que, en algún momento, se entregará un valor o se informará un error. El código puede continuar su ejecución normalmente, mientras "espera" que esa promesa se cumpla o se rechace.

Este paradigma permite a JavaScript trabajar de forma no bloqueante y escalar en complejidad sin comprometer la fluidez ni la estabilidad de la aplicación. Por eso, **la adopción de promesas es una práctica estándar en la actualidad**, especialmente cuando se trabaja con APIs modernas o se desarrollan interfaces que dependen de múltiples operaciones asincrónicas.

Creación y uso de promesas personalizadas.

Hasta este punto, se ha trabajado con promesas desde una perspectiva de consumo: se utilizan cuando una API o función ya retorna una promesa. Sin embargo, un aspecto clave para dominar verdaderamente este modelo es la capacidad de crear promesas personalizadas, es decir, funciones asincrónicas

propias que internamente utilizan promesas para manejar resultados futuros. Esta habilidad permite estructurar el código con mayor claridad, encapsular procesos complejos y reutilizar lógica asíncronica de manera ordenada y predecible.

En concreto, una promesa personalizada se construye a partir de la clase nativa **Promise** de JavaScript. Esta clase recibe como argumento una función que, a su vez, recibe dos parámetros: **resolve** y **reject**. El primero se invoca cuando la operación se completa exitosamente y el segundo cuando se produce un error o una condición que impide continuar.

En términos generales, el uso de promesas personalizadas resulta especialmente útil cuando se necesita integrar comportamientos asíncronos que no están nativamente basados en promesas, como ciertas APIs del navegador (por ejemplo, `FileReader`, geolocalización o eventos), funciones que utilizan *callbacks* tradicionales o cualquier lógica interna que requiera esperar un resultado antes de continuar. Al envolver estas operaciones en una promesa, se gana control total sobre el flujo del programa, facilitando la lectura del código y su integración con estructuras más modernas como **async/await**. Sin duda, esta capacidad amplía significativamente el poder expresivo de JavaScript en el manejo de procesos asíncronos.

Estructura básica de una promesa personalizada.

La estructura general para crear una promesa es la siguiente:

```
const miPromesa = new Promise((resolve, reject) => {  
  // operación asíncronica simulada  
  if (/* condición de éxito */) {  
    resolve(valor);  
  } else {  
    reject(error);  
  }  
});
```

Una vez creada, esta promesa se comporta igual que cualquier otra: puede ser consumida con `.then()` y `.catch()`.

Este patrón es útil, por ejemplo, cuando se desea simular una espera, validar datos de forma diferida, o interactuar con APIs que aún no utilizan promesas nativamente.

Ejemplo con tiempo de espera simulado.

Imaginemos una función que simula cargar datos del servidor con un retardo:

```
function cargarDatos() {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      const exito = true; // Simula una respuesta del servidor  
  
      if (exito) {  
        resolve(["dato1", "dato2", "dato3"]);  
      } else {  
        reject("Error al cargar los datos");  
      }  
    }, 2000);  
  });  
}
```

Esta función no devuelve inmediatamente los datos, sino una promesa de que los entregará más adelante. Desde el punto de vista del consumidor, se puede usar de la siguiente manera:

```
cargarDatos()  
  .then(datos => console.log("Datos recibidos:", datos))  
  .catch(error => console.error("Ocurrió un error:", error));
```

Ante esto, el consumidor no necesita saber cómo funciona internamente la lógica asincrónica. De hecho, solo debe manejar el resultado cuando llegue. En este sentido, esta **abstracción del tiempo y del detalle técnico** es uno de los mayores beneficios del uso de promesas personalizadas.

Ventajas del uso de promesas propias.

Crear promesas personalizadas ofrece múltiples ventajas:

- ✓ **Encapsulan lógica compleja:** pueden envolver procesos que involucran varias validaciones, cálculos u otras operaciones asincrónicas.
- ✓ **Facilitan el *testing* y la reutilización:** al separar la lógica en funciones autónomas que retornan promesas, es más fácil probarlas y usarlas en diferentes partes de la aplicación.
- ✓ **Preparan el código para integrar *async/await*:** las funciones que retornan promesas son compatibles con la sintaxis moderna basada en *async*, facilitando una transición progresiva hacia patrones de código más avanzados.
- ✓ **Permiten convertir APIs antiguas basadas en *callbacks* en promesas:** esto es especialmente útil cuando se trabaja con bibliotecas heredadas.

Conversión de una función con *callbacks* a promesas.

Una de las aplicaciones más prácticas de este conocimiento es convertir funciones que originalmente usaban *callbacks* en versiones que retornan promesas. Este patrón es conocido como **promisificación** (*promisify*).

Supongamos que se dispone de esta función tradicional basada en *callbacks*:

```
function obtenerUsuario(callback) {  
  setTimeout(() => {  
    callback(null, { nombre: "Ana", edad: 30 });  
  }, 1000);  
}
```

Dicha función puede transformarse de la siguiente manera:

```
function obtenerUsuarioPromesa() {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      const usuario = { nombre: "Ana", edad: 30 };  
      resolve(usuario);  
    }, 1000);  
  });  
}
```

Esto habilita su uso con `.then()` y `.catch()`, o incluso con `await`, permitiendo una integración más moderna y mantenible.

Buenas prácticas en la creación de promesas.

- Siempre envolver solo el código asíncrono dentro de la función **new Promise**. Evitar mezclar lógica síncrona innecesaria.
- Invocar **resolve** y **reject** una única vez. Lllamarlos más de una vez genera comportamiento impredecible.
- No olvidar capturar y propagar los errores. En operaciones más complejas, se recomienda usar bloques **try/catch** dentro del ejecutor de la promesa.
- Retornar la promesa para que quien la invoque pueda encadenarla correctamente.

Consideraciones de diseño.

Si bien es posible crear promesas para cualquier tipo de operación, se recomienda hacerlo únicamente cuando:

- Se trabaja con código asíncrono (tiempos de espera, acceso a recursos externos, eventos que pueden demorar).
- Se requiere encapsular lógica que se comporta como una operación a futuro, y no puede o no debe ejecutarse de inmediato.

En efecto, crear promesas en operaciones puramente sincrónicas carece de sentido práctico y puede generar complejidad innecesaria.

Funciones asincrónicas con `async` y `await`.

El modelo de promesas introdujo una forma más ordenada y estructurada de trabajar con operaciones asincrónicas. Sin embargo, a medida que crecen la cantidad de pasos encadenados, el uso intensivo de `.then()` y `.catch()` puede volver el código nuevamente difícil de leer, en especial cuando intervienen condicionales, estructuras de control o múltiples funciones que dependen entre sí.

Para resolver este problema, JavaScript incorporó en ES2017 (ES8) la sintaxis basada en **funciones asincrónicas con `async` y `await`**; una evolución natural del modelo de promesas. Esta sintaxis permite escribir código asincrónico con una estructura **similar a la del código sincrónico**, conservando su claridad y legibilidad, pero sin bloquear el hilo principal.

¿Qué es una función `async`?

Una función declarada con la palabra clave `async` es una función especial que, de forma automática, **retorna una promesa**. Esto significa que cualquier valor retornado dentro de ella será automáticamente envuelto en una promesa resuelta. Además, dentro de su cuerpo se pueden utilizar expresiones `await` para **pausar temporalmente la ejecución** hasta que una promesa se resuelva.

En el siguiente ejemplo, se presenta una función que, aunque simplemente devuelve una cadena, lo hace como una promesa. Esto permite tratarla como una operación asincrónica desde el exterior:

```
async function obtenerDatos() {  
  return "Resultado";  
}  
  
obtenerDatos().then(res => console.log(res)); //  
"Resultado"
```

¿Qué hace await?

La palabra clave **await** solo puede usarse dentro de funciones **async**, y su propósito es **esperar la resolución de una promesa** antes de continuar con la siguiente línea de código. Mientras se espera, el motor de JavaScript **no bloquea** el resto del programa; simplemente **suspende temporalmente** la ejecución de la función asincrónica.

```
async function mostrarDatos() {  
  const respuesta = await fetch("https://api.ejemplo.com/datos");  
  const datos = await respuesta.json();  
  console.log(datos);  
}
```

Este código tiene el mismo efecto que una cadena de **.then()**, pero resulta mucho más legible y mantiene una estructura más cercana a la de un lenguaje imperativo clásico. Gracias a **await**, el flujo de lectura es lineal y fácil de seguir: se pide una respuesta, se espera, se transforma, y luego se utiliza.

Ventajas de await.

- **Mayor claridad en flujos complejos**, especialmente cuando se necesita realizar varias operaciones asincrónicas en serie o condicionales.
- **Integración más limpia con estructuras de control** como `if`, `try/catch`, `for`, etc.
- **Manejo centralizado de errores** mediante `try/catch`, similar al paradigma sincrónico.
- **Facilidad de mantenimiento** en proyectos medianos o grandes, donde la legibilidad del código es clave.

Manejo de errores con try/catch.

Al igual que en el modelo tradicional de programación sincrónica, el uso de bloques `try/catch` permite capturar errores que se producen en cualquier punto de la función `async`.

```
async function procesar() {  
  try {  
    const respuesta = await fetch("https://api.inexistente.com");  
    const datos = await respuesta.json();  
    console.log(datos);  
  } catch (error) {  
    console.error("Error al procesar los datos:", error);  
  }  
}
```

Este enfoque **centraliza el manejo de errores**, haciendo el código más robusto y profesional, especialmente cuando se trabaja con fuentes externas que pueden fallar (APIs, archivos, redes, dispositivos).

Ejemplo comparativo: then vs await.

Con `.then()`:

```
fetch("https://api.ejemplo.com")  
  .then(res => res.json())  
  .then(data => console.log(data))  
  .catch(err => console.error(err));
```

Con `async/await`:

```
async function cargar() {  
  try {  
    const res = await fetch("https://api.ejemplo.com");  
    const data = await res.json();  
    console.log(data);  
  } catch (err) {  
    console.error(err);  
  }  
}
```

Ambos fragmentos hacen lo mismo, pero la versión con `async/await` es más clara cuando se agregan múltiples pasos, validaciones o condicionales.

Además, favorece el uso de **nombres descriptivos y flujos controlados**, algo clave en proyectos colaborativos o de larga duración.

Consideraciones importantes.

- El uso de `await` detiene temporalmente la ejecución de la función `async`, pero no bloquea el resto del programa. Esto es gestionado por el `event loop`.
- Si una expresión `await` lanza un error y no se captura con `try/catch`, el programa generará una excepción no controlada.
- Es posible usar `await` dentro de bucles, pero con precaución, ya que cada iteración esperará a la anterior. Para ejecuciones paralelas, se recomienda usar `Promise.all`.

Errores comunes y buenas prácticas en asincronía.

Si bien el uso de asincronía en JavaScript es indispensable para el desarrollo moderno, también puede introducir **complejidades difíciles de detectar**. En efecto, las operaciones diferidas, las respuestas impredecibles del entorno externo, y la interacción entre múltiples procesos asíncronos obligan al programador a tomar decisiones técnicas cuidadosamente diseñadas.

Este apartado analiza los errores más frecuentes que suelen cometerse al implementar asincronía con promesas o `async/await`, y presenta un conjunto de **buenas prácticas fundamentales** para asegurar un desarrollo confiable, legible y mantenible.

Errores comunes en asincronía.

1. **No retornar la promesa en funciones intermedias.** Cuando se utilizan funciones que internamente hacen uso de promesas, es frecuente olvidar

el **return**. Esto impide que la promesa sea encadenada correctamente, rompiendo el flujo de ejecución y dificultando el manejo de errores.

```
function obtenerDatos() {  
  fetch("/api/datos") // ❌ Falta el return  
  .then(res => res.json());  
}
```

Solución:

```
function obtenerDatos() {  
  return fetch("/api/datos") // ✅ Promesa correctamente retornada  
  .then(res => res.json());  
}
```

2. **Manejo incorrecto de errores en `async/await`**. Es común suponer que el uso de **`await`** exime al código de manejar fallos. Sin embargo, si una promesa es rechazada dentro de una función **`async`** sin bloque **`try/catch`**, el error no será capturado y puede generar un fallo crítico.

```
async function cargarDatos() {  
  const res = await fetch("/api/invalido"); // ❌ Si falla, rompe la ejecución  
  const datos = await res.json();  
}
```

Solución:

```
async function cargarDatos() {  
  try {  
    const res = await fetch("/api/invalido");  
    const datos = await res.json();  
    return datos;  
  } catch (error) {  
    console.error("Error al cargar datos:", error);  
    return null;  
  }  
}
```

3. Usar **await** dentro de bucles sin necesidad. Cada llamada a **await** dentro de un bucle detiene la ejecución de la iteración siguiente hasta que la anterior finaliza, lo que puede degradar el rendimiento cuando no es necesario.

```
for (const id of ids) {  
  const data = await fetch(`/api/usuario/${id}`); // ❌ Secuencialmente lento  
}
```

Solución: ejecutar en paralelo si no hay dependencias entre iteraciones.

```
const promesas = ids.map(id => fetch(`/api/usuario/${id}`));  
const respuestas = await Promise.all(promesas); // ✅ Más rápido
```

4. Encadenar **then** y usar **await** innecesariamente juntos. Combinar **.then()** con **await** genera un código innecesariamente redundante y más difícil de seguir.

```
const respuesta = await fetch("/api/datos").then(res => res.json()); // ✗ innecesario
```

Buenas prácticas recomendadas.

1. **Siempre manejar los errores explícitamente.** El desarrollo asincrónico debe contemplar siempre el fracaso como posibilidad. Es buena práctica capturar errores con `catch()` o bloques `try/catch`, incluso cuando se trabaja en prototipos.
2. **Centralizar el manejo de errores.** En lugar de manejar errores repetidamente en cada punto del flujo, se recomienda encapsular la lógica y concentrar los errores en un solo lugar, mejorando la trazabilidad y reduciendo código repetido.
3. **Evitar el uso excesivo de `setTimeout` como solución universal.** Aunque útil, `setTimeout` no garantiza una ejecución precisa y puede ocultar problemas de sincronización real. Es preferible trabajar con promesas reales o APIs diseñadas para asincronía.
4. **No mezclar innecesariamente `async/await` con `.then()`.** Ambos estilos son compatibles, pero deben usarse de forma coherente para facilitar la lectura y depuración.
5. **Usar `Promise.all` o `Promise.allSettled` para concurrencia controlada.** Cuando se desea ejecutar múltiples operaciones en paralelo, estas funciones permiten optimizar el rendimiento y manejar los resultados de forma agregada, especialmente útil en integraciones con APIs múltiples.

6. **Documentar claramente las funciones que retornan promesas.** Una buena práctica profesional es anotar en los comentarios o documentación cuándo una función retorna una promesa o requiere `await`, para facilitar su uso correcto por otros desarrolladores.

Clases, métodos avanzados de arrays.

Programación orientada a objetos en JavaScript.

La programación orientada a objetos (POO) es uno de los paradigmas más utilizados en el desarrollo de *software* debido a su capacidad para representar entidades complejas mediante estructuras llamadas objetos. Este enfoque resulta especialmente útil cuando se busca modelar situaciones del mundo real dentro de una aplicación, ya que permite agrupar información y funcionalidades relacionadas dentro de unidades lógicas coherentes.

En su forma más general, un objeto es una colección de propiedades y métodos que describen su estado y comportamiento. Bajo este marco, JavaScript, a diferencia de lenguajes como Java o C#, se basa en un modelo de objetos **prototípico** y no en clases tradicionales. Sin embargo, con la introducción de **ECMAScript 6 (ES6)**, se incorporó una sintaxis de clases que abstrae el modelo prototípico y facilita la escritura de código orientado a objetos de forma más clara y estandarizada.

En definitiva, la POO en JavaScript permite construir programas más organizados, mantener el código reutilizable, y definir relaciones jerárquicas entre objetos, facilitando la evolución y escalabilidad de las aplicaciones.

Clases como estructuras moldeadoras.

Una **clase** en JavaScript actúa como un plano o plantilla para la creación de objetos. En ella se define qué atributos tendrá cada objeto creado (a través del constructor), y qué acciones podrá realizar (a través de métodos). Ante esto es

importante remarcar que, aunque la sintaxis de clases resulta familiar para quienes provienen de otros lenguajes, en JavaScript esto continúa siendo una abstracción sobre su modelo interno basado en prototipos.

En detalle, las clases permiten instanciar múltiples objetos similares, cada uno con sus propios valores pero con un comportamiento compartido. Esta propiedad es esencial cuando se desea mantener coherencia en la lógica de objetos similares (por ejemplo, usuarios, productos, pedidos, etc.).

```
class Producto {  
  constructor(nombre, precio) {  
    this.nombre = nombre;  
    this.precio = precio;  
  }  
  
  obtenerDescripcion() {  
    return `Producto: ${this.nombre} - Precio: ${this.precio}`;  
  }  
}
```

Constructores y métodos.

El **constructor** es un método especial que se ejecuta automáticamente al crear una instancia de la clase y su función es inicializar las propiedades del objeto. A su vez, los **métodos** definen las acciones que el objeto puede realizar, es decir, el comportamiento que se asocia a su lógica interna.

En concreto, un objeto instanciado a partir de una clase mantiene sus propios datos internos pero comparte la definición de métodos con todos los demás objetos creados desde esa clase. Esto optimiza el uso de memoria y centraliza la lógica de comportamiento.

```
const producto1 = new Producto("Auriculares", 15000);  
console.log(producto1.obtenerDescripcion());
```

Encapsulamiento y organización del código.

Uno de los principios fundamentales de la POO es el **encapsulamiento**, que permite ocultar detalles de implementación y exponer solo lo necesario para interactuar con el objeto. En JavaScript moderno, este encapsulamiento puede lograrse mediante convenciones de nomenclatura, propiedades privadas (#) o mediante cierres (*closures*), aunque aún no existe un sistema de control de acceso tan estricto como en otros lenguajes.

En definitiva, la encapsulación favorece la modularidad del sistema, reduce la dependencia entre componentes, y previene errores derivados de la manipulación directa de estados internos no controlados.

Herencia y reutilización.

La **herencia** es el mecanismo por el cual una clase puede extender otra, heredando sus propiedades y comportamientos. Esto permite **crear jerarquías lógicas** en la estructura del código, evitando la duplicación y favoreciendo la reutilización de lógica compartida. En JavaScript, esto se logra con la palabra clave **extends**.

Por su parte, el método **super** permite acceder a la implementación de la clase madre desde la clase hija, facilitando la extensión de funcionalidades sin necesidad de reescribirlas.

```
class ProductoDigital extends Producto {  
  constructor(nombre, precio, formato) {  
    super(nombre, precio);  
    this.formato = formato;  
  }  
  
  obtenerDescripcion() {  
    return `${super.obtenerDescripcion()} - Formato: ${this.formato}`;  
  }  
}
```

Relevancia en el desarrollo *fullstack*.

El paradigma orientado a objetos se encuentra presente tanto en el desarrollo *frontend* como *backend*. En el *frontend*, permite modelar componentes reutilizables, representar entidades de negocio, o encapsular comportamientos en clases que gestionan interfaces o interacciones complejas. En el *backend*, se usa para estructurar modelos de base de datos, servicios, controladores y lógica de negocio.

Incluso, en librerías modernas como React, aunque no siempre se utilizan clases explícitas, el enfoque orientado a componentes mantiene muchos de los principios de la POO: encapsulamiento, reutilización, composición y herencia a través de jerarquías lógicas.

Métodos avanzados de arrays en JavaScript.

El desarrollo moderno con JavaScript ha evolucionado hacia un estilo de programación más **declarativo**, donde se privilegia **qué se quiere hacer** con

los datos por sobre **cómo se hace**. En este contexto, los métodos avanzados de arrays se presentan como herramientas clave que permiten expresar de forma concisa, legible y precisa las operaciones más habituales sobre colecciones.

Estos métodos no solo ayudan a escribir menos código, sino que lo hacen más expresivo, más fácil de testear y de mantener. A la par, al evitar bucles explícitos y mutaciones del estado, favorecen patrones más funcionales que son altamente valorados en la industria.

Ventajas del enfoque declarativo.

A diferencia de los bucles tradicionales (**for**, **while**), los métodos como **map**, **filter** o **reduce** permiten expresar una intención clara en una única línea de código. Con estos métodos no se describe el mecanismo de iteración (contador, condición, incremento), sino el objetivo: transformar, filtrar, acumular, buscar.

Este enfoque presenta las siguientes ventajas a tener en cuenta:



Reduce el riesgo de errores derivados de índices mal manejados.



Evita efectos secundarios y promueve funciones puras.



Facilita la composición de funciones, permitiendo construir pipelines de operaciones.



Favorece la lectura del código, especialmente en revisiones o trabajo en equipo.

La importancia de `reduce()` en operaciones complejas.

Entre todos los métodos avanzados, `reduce()` destaca por su versatilidad. Aunque puede parecer complejo al principio, su verdadero poder radica en que permite condensar estructuras completas a partir de una acumulación progresiva. En específico, con `reduce`, se pueden implementar patrones como:

- Sumar o concatenar valores.
- Agrupar elementos por categoría.
- Convertir una lista en un objeto.
- Contar ocurrencias de un elemento.
- Simular estructuras como mapas o diccionarios.

Ejemplo conceptual: agrupar nombres por su primera letra.

```
const nombres = ["Ana", "Andrés", "Bruno", "Belén"];
const agrupados = nombres.reduce((acc, nombre) => {
  const letra = nombre[0];
  if (!acc[letra]) acc[letra] = [];
  acc[letra].push(nombre);
  return acc;
}, {});
```

Este tipo de uso muestra cómo `reduce` permite modelar estructuras complejas y personalizadas que serían difíciles de construir con otros métodos.

Mutabilidad vs inmutabilidad.

Los métodos avanzados como **map**, **filter** o **reduce** no modifican el array original, sino que retornan uno nuevo. Esta ventaja es importante en términos de diseño de *software*, ya que favorece la **inmutabilidad**, un principio central en la programación funcional.

En general, evitar la mutación directa de los datos reduce efectos colaterales inesperados, especialmente en aplicaciones con múltiples componentes que acceden o dependen del mismo conjunto de datos.

Aplicaciones reales en proyectos *fullstack*.

El uso de estos métodos es habitual tanto en el *frontend* como en el *backend*:

- **En el *frontend***, se utilizan para renderizar listas dinámicas, aplicar filtros a datos visualizados, transformar la información recibida desde una API en una estructura útil para el usuario o agrupar datos para visualizaciones gráficas.
- **En el *backend***, son comunes en operaciones de limpieza o procesamiento de datos antes de guardar en una base de datos, en generación de reportes, validación de información o transformación de objetos para enviar como respuesta HTTP.

Métodos más utilizados en arrays.

forEach() – Recorrer sin modificar.

Este método ejecuta una función por cada elemento del array. Es útil cuando se desea realizar acciones con efectos colaterales, como imprimir, registrar o modificar valores externos, pero **no retorna un nuevo array**.

```
const productos = ["Mesa", "Silla", "Lámpara"];
productos.forEach((producto, index) => {
  console.log(`Producto ${index + 1}: ${producto}`);
});
```

map() – Transformar sin mutar.

map() crea un nuevo array con los resultados de aplicar una función a cada elemento. Es ideal para transformar datos sin alterar el original.

```
const precios = [100, 200, 300];
const conDescuento = precios.map(precio => precio * 0.9);
// Resultado: [90, 180, 270]
```

filter() – Seleccionar por condición.

filter() devuelve un nuevo array solo con los elementos que cumplan una condición. No modifica el array original.

```
const edades = [12, 17, 19, 24, 16];
const mayoresDeEdad = edades.filter(edad => edad >= 18);
// Resultado: [19, 24]
```

find() – Buscar el primero que cumpla.

A diferencia de **filter**, **find()** devuelve **un único elemento** (el primero que cumpla la condición), o **undefined** si ninguno la cumple.

```
const usuarios = [
  { nombre: "Ana", activo: false },
  { nombre: "Luis", activo: true }
];

const usuarioActivo = usuarios.find(u => u.activo);
// Resultado: { nombre: "Luis", activo: true }
```

some() – ¿Al menos uno cumple?

some() verifica si **al menos un elemento** del array cumple una condición. Devuelve un booleano (**true** o **false**).

```
const valores = [0, null, "", 42];
const hayVerdadero = valores.some(valor => Boolean(valor));
// Resultado: true
```




Bibliografía utilizada y sugerida

JavaScript.info. (2025). *The Modern JavaScript Tutorial*. <https://javascript.info>¹

CSS-Tricks. (s. f.). *JavaScript*. <https://css-tricks.com/tag/javascript/>²

MDN Web Docs. (s. f.). *JavaScript*. Mozilla Corporation.
<https://developer.mozilla.org/en-US/docs/Web/JavaScript>³

Stack Overflow. (s. f.). *[javascript]*.
<https://stackoverflow.com/questions/tagged/javascript>⁴

W3Schools. (s.f.). *JavaScript Tutorial*. <https://www.w3schools.com/js/>⁵

¹ Sitio educativo completo, ideal para quienes comienzan con JavaScript. Presenta explicaciones detalladas y ejemplos claros sobre programación orientada a objetos, asincronía, arrays y mucho más.

² Aunque enfocado inicialmente en CSS, este sitio contiene numerosos artículos sobre manipulación de arrays, asincronía con **async/await**, promesas y conceptos avanzados de JavaScript.

³ Documentación oficial mantenida por Mozilla. Es la fuente más confiable para consultar métodos de arrays, clases, funciones asincrónicas y promesas. Incluye definiciones técnicas, ejemplos y notas de compatibilidad.

⁴ Espacio colaborativo para resolver dudas concretas. Aunque no es un recurso educativo estructurado, permite aprender mediante ejemplos reales, soluciones optimizadas y debate técnico entre programadores.

⁵ Portal interactivo de aprendizaje con ejemplos prácticos, ejercicios editables y referencias básicas para cada método. Su enfoque paso a paso lo convierte en un recurso introductorio útil.